

Tarea 3: Procesamiento de Imágenes en CUDA

Javier Torres & Juan Umaña & Benjamin Jimenez

Repositorio: <https://github.com/khewa11/T3-Paralel>

Sistemas Distribuidos y Paralelos
Universidad de Concepción
Junio 2026

1. Introducción

El análisis estadístico de grandes colecciones de imágenes constituye un pilar fundamental en disciplinas como la visión por computador, la compresión con pérdida y el aprendizaje de representaciones. Una operación recurrente en estos contextos es el cálculo de la **matriz de covarianza** a partir de un conjunto de N imágenes aplanadas en vectores de dimensión D , donde $D = W \times H \times C$ representa el producto entre el ancho, el alto y el número de canales de cada imagen.

1.1. Formulación matemática

Sea $\{v^{(k)}\}_{k=1}^N$ el conjunto de imágenes aplanadas, con $v^{(k)} \in \mathbb{R}^D$. El procesamiento se descompone en tres etapas:

1. **Vector promedio.** Se calcula el centroide del conjunto:

$$\mu_j = \frac{1}{N} \sum_{k=1}^N v_j^{(k)}, \quad j = 1, \dots, D. \quad (1)$$

2. **Matriz centrada.** Cada vector se centra restando el promedio:

$$\bar{v}_j^{(k)} = v_j^{(k)} - \mu_j, \quad k = 1, \dots, N, \quad j = 1, \dots, D. \quad (2)$$

3. **Matriz de covarianza.** Se obtiene la matriz simétrica $\mathbf{C} \in \mathbb{R}^{D \times D}$:

$$C_{jj'} = \frac{1}{N-1} \sum_{k=1}^N \bar{v}_j^{(k)} \bar{v}_{j'}^{(k)}, \quad j, j' = 1, \dots, D. \quad (3)$$

1.2. Motivación y limitante física

El desafío computacional no reside únicamente en la complejidad $\mathcal{O}(N \cdot D^2)$ del producto exterior acumulado, sino en la **limitante física de la memoria de la GPU**. Aplanar miles de imágenes del dataset DIV2K en una única región contigua de la VRAM excede con creces la capacidad de tarjetas gráficas de gama media. Por ejemplo, una GPU con aproximadamente 3.7 GB de VRAM no puede alojar simultáneamente la matriz de datos completa junto con la matriz de covarianza resultante cuando las resoluciones se escalan más allá de umbrales modestos.

Este trabajo propone y evalúa una estrategia de **orquestración asíncrona mediante CUDA Streams** que particiona los datos en lotes (*batches*), permitiendo procesar volúmenes de datos que superan la capacidad física de la VRAM. Se compara esta implementación con una versión tradicional que opera sobre el *Stream 0* por defecto, y se analizan las implicaciones del *latency hiding* sobre el bus PCIe.

2. Descripción de Algoritmos

2.1. Implementación Tradicional (Stream 0)

La primera implementación sigue el paradigma convencional de programación CUDA:

- **Memoria contigua:** Todas las imágenes se alojan en un único buffer de memoria global de la GPU mediante `cudaMalloc`. Igualmente, la matriz de covarianza de tamaño $D \times D$ se aloja íntegramente en la VRAM.
- **Stream único (Stream 0):** Todas las transferencias *Host* $\rightarrow$ *Device* y las invocaciones de *kernels* se serializan en el flujo por defecto, sin posibilidad de solapamiento entre cómputo y comunicación.
- **Tiling:** El cálculo de la covarianza emplea *tiling* en memoria compartida para maximizar la reutilización de datos dentro de cada bloque de hilos, reduciendo los accesos a memoria global.

Limitación crítica: Esta implementación falla con un error de **Out of Memory** al escalar la resolución de las imágenes (por ejemplo, a 112×84 píxeles) debido a que el volumen total de datos excede la capacidad de la VRAM disponible (3715 MB). La asignación simultánea de la matriz de datos y la matriz de covarianza resulta inviable bajo estas condiciones.

2.2. Orquestación con CUDA Streams

La segunda implementación resuelve la limitación anterior mediante una estrategia de particionamiento por lotes con orquestación asíncrona:

- **Particionamiento en lotes (*batches*):** El conjunto de N imágenes se divide en S lotes de tamaño $\lceil N/S \rceil$. Cada lote se procesa de forma independiente, acumulando su contribución parcial a la matriz de covarianza.
- **Pinned Memory (`cudaMallocHost`):** Los buffers del lado del host se alojan como memoria *page-locked*, habilitando transferencias DMA que no requieren una copia intermedia al *staging buffer* del sistema operativo. Esto es requisito indispensable para utilizar `cudaMemcpyAsync`.
- **Transferencias asíncronas (`cudaMemcpyAsync`):** Las copias entre *Host* y *Device* se emiten de forma no bloqueante, permitiendo que el motor de copia DMA opere en paralelo con la ejecución de *kernels* en otro *stream*.
- **Ocultamiento de latencia (*Latency Hiding*):** Al solapar la transferencia del lote $i + 1$ con el cómputo del lote i , se busca ocultar la latencia del bus PCIe y reducir el tiempo total de ejecución.

La acumulación de resultados parciales se realiza directamente sobre la matriz de covarianza en la GPU, garantizando que el resultado final sea numéricamente idéntico al de la implementación tradicional.

3. Metodología Experimental

3.1. Hardware

La Tabla 1 resume las características del sistema utilizado para los experimentos.

Tabla 1: Especificaciones del hardware experimental.

Componente	Especificación
CPU	Intel Core i3-1005G1 @ 1.20 GHz
RAM	12 GB DDR4
GPU	NVIDIA GeForce GTX 1650 Ti
VRAM	3715 MB (4 GB)
Bus	PCIe 3.0 x16
SO	Linux Mint 21.3
CUDA	Toolkit 12.x (sm_75)

3.2. Dataset

Se utilizó el dataset **DIV2K**, una colección de referencia compuesta por imágenes de alta resolución ampliamente empleada en tareas de super-resolución. Para los experimentos, las imágenes fueron redimensionadas y aplanadas, generando un volumen de datos de aproximadamente 3 MB para el conjunto de prueba principal.

3.3. Parámetros del experimento

El parámetro principal de variación fue el **número de streams** S , evaluado en el conjunto $S \in \{1, 2, 4, 8, 16\}$. La configuración con $S = 1$ representa el caso base en el que la orquestación por lotes no aporta solapamiento efectivo, funcionando de manera análoga a una ejecución serializada dentro de la infraestructura de *streams*.

4. Resultados

4.1. Resultados del Experimento 1

El Experimento 1 corresponde a la implementación tradicional sobre el Stream 0, en la cual toda la memoria se aloja de forma monolítica en la VRAM. La Tabla 2 muestra los tiempos obtenidos para tres resoluciones.

Tabla 2: Tiempos de ejecución (ms) de la implementación tradicional (Stream 0) para distintas resoluciones.

Resolución	$n = W \times H \times C$	H→D (ms)	Cómputo (ms)	D→H (ms)	Total (ms)
64×48	9 216	0.74	100.79	51.41	152.93
96×72	20 736	1.47	1 007.95	540.12	1 549.55
112×84	28 224		<i>Out of Memory</i>		

Se observa un crecimiento significativo del tiempo de cómputo y de la copia *Device* → *Host* al pasar de 64×48 a 96×72 , consistente con la complejidad cuadrática $\mathcal{O}(N \cdot D^2)$ del producto exterior acumulado. Al intentar procesar imágenes de 112×84 píxeles, la implementación tradicional falla con un error de **Out of Memory**, ya que la asignación monolítica de la matriz de datos y la matriz de covarianza excede la capacidad de la VRAM (3715 MB).

4.2. Comparación entre experimentos

La Tabla 3 confronta los tiempos totales del Experimento 1 (Stream 0) con los del Experimento 2 (CUDA Streams, $S = 4$) para las tres resoluciones evaluadas.

Tabla 3: Comparación de tiempos totales (ms) entre ambos experimentos.

Resolución	Exp. 1 — Stream 0 (ms)	Exp. 2 — Streams $S=4$ (ms)
64×48	152.9	1 997.7
96×72	1 549.6	3 510.3
112×84	OOM	3 524.8

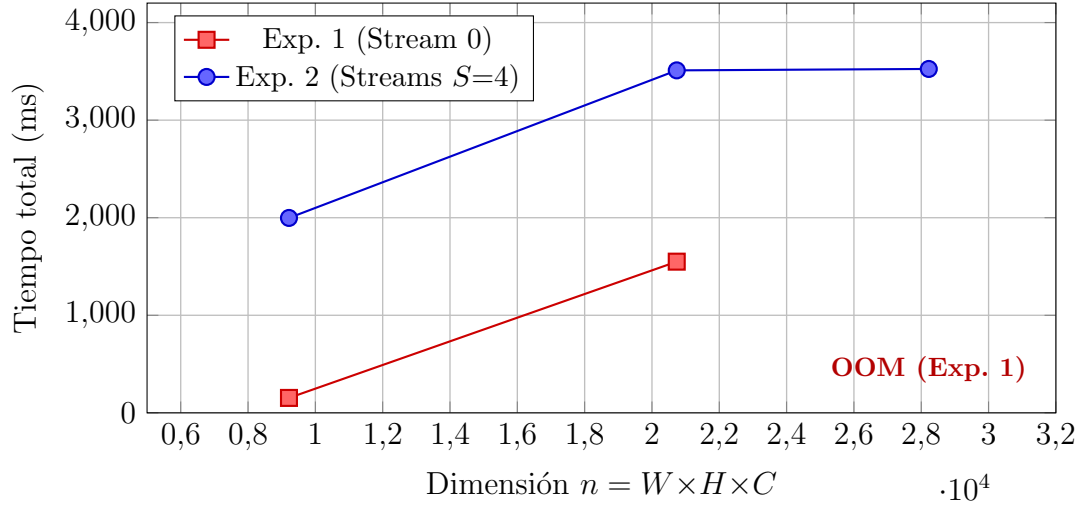


Figura 1: Tiempo total en función de la dimensión n para ambos experimentos. El Experimento 1 no puede procesar $n = 28\,224$ (OOM), mientras que la orquestación por lotes del Experimento 2 completa todas las resoluciones.

A continuación se presentan los resultados detallados del Experimento 2, correspondiente al procesamiento del dataset de 3 MB con la implementación basada en CUDA Streams. Se reportan los tiempos de ejecución del *kernel* en la GPU, el costo de la copia *Device* $\rightarrow$ *Host* y el tiempo total para cada configuración de S .

4.3. Tiempos del Experimento 2 variando S

Tabla 4: Tiempos de ejecución (ms) variando el número de streams S sobre el dataset de 3 MB.

Streams (S)	GPU (ms)	Copia D $\rightarrow$ H (ms)	Total (ms)
1	2478.59	60.97	2539.56
2	2615.94	67.06	2683.00
4	2365.07	55.60	2420.67
8	2460.84	57.05	2517.89
16	2514.05	57.88	2571.94

4.4. Análisis de Speedup

Se define el *speedup* relativo tomando como referencia la configuración con $S = 1$:

$$\text{Speedup}(S) = \frac{T_{\text{total}}(S = 1)}{T_{\text{total}}(S)}. \quad (4)$$

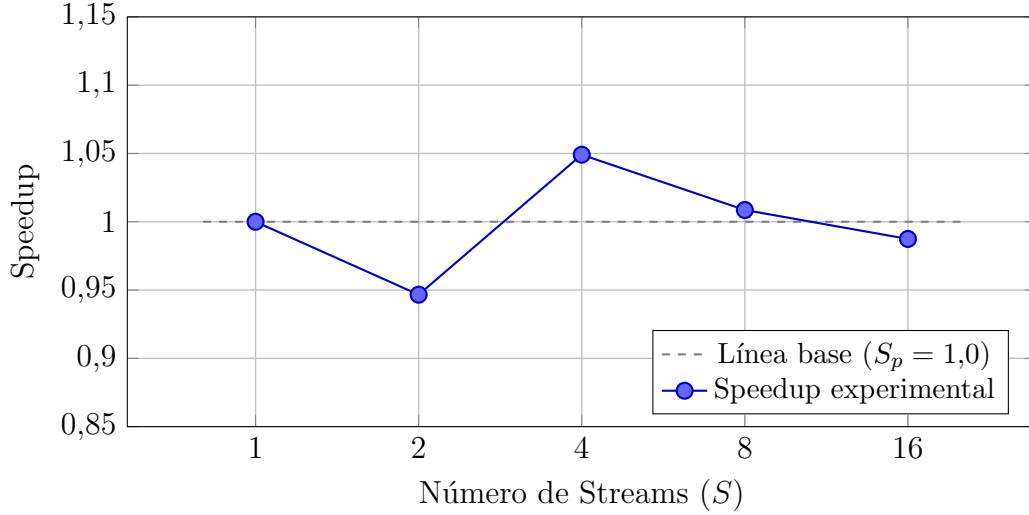


Figura 2: Speedup en función del número de streams. La curva experimental oscila en torno a la línea base, evidenciando la ausencia de mejora significativa por solapamiento.

4.5. Perfiles de ejecución (Nsight Systems)

Los perfiles capturados con Nsight Systems para la configuración $S = 4$ sobre el dataset de 3 MB permiten descomponer el tiempo de ejecución a nivel de llamadas CUDA y kernels individuales.

Tabla 5: Distribución del tiempo por llamada CUDA API (Nsight Systems, $S = 4$, resolución 96×72).

Llamada CUDA API	Tiempo (ms)	% del total
cudaHostAlloc	121.93	26.5
cudaStreamSynchronize	113.12	24.6
accumulateCovKernel<16>	112.81	24.5
cudaMemcpy (total)	52.89	11.5
D→H	52.74	11.5
H→D	1.18	0.3
cudaMemset	1.81	0.4

Tabla 6: Desglose de kernels ejecutados en la GPU (Nsight Systems, $S = 4$, resolución 96×72).

Kernel	Instancias	Tiempo (ms)	% GPU
<code>accumulateCovKernel<16></code>	4	112.81	99.9
<code>centerBatchKernel</code>	4	0.069	0.1
<code>accumulateMeanKernel</code>	4	0.026	<0.1
<code>finalizeMeanKernel</code>	1	0.002	<0.1

Del análisis del perfil se extraen tres hallazgos clave:

1. El kernel `accumulateCovKernel<16>` concentra el **99.9 %** del tiempo de GPU, confirmando que el cálculo de la covarianza es la operación dominante.
2. La transferencia *Host* $\rightarrow$ *Device* (1.18 ms) es insignificante frente al cómputo, mientras que la copia *Device* $\rightarrow$ *Host* (52.74 ms) refleja el costo de extraer la matriz de covarianza resultante.
3. Las llamadas de infraestructura (`cudaHostAlloc` y `cudaStreamSynchronize`) representan conjuntamente más del 50 % del tiempo de la API, un overhead significativo que no contribuye al cómputo útil.

La razón entre la transferencia de entrada y el cómputo del kernel dominante resulta:

$$\frac{T_{H \rightarrow D}}{T_{\text{cov}}} = \frac{1,18}{112,81} \approx 1,0 \%. \quad (5)$$

5. Discusión de Resultados y Análisis Crítico

Los resultados experimentales revelan un comportamiento aparentemente paradójico: la orquestación mediante múltiples CUDA Streams no produjo una reducción significativa del tiempo total de ejecución. Sin embargo, un análisis riguroso demuestra que este resultado es, de hecho, el comportamiento esperado dada la naturaleza del cómputo involucrado, y que la verdadera contribución de la arquitectura por lotes reside en un plano diferente al de la aceleración temporal.

5.1. Cuello de botella: dominio del cómputo

La Tabla 4 permite cuantificar la distribución del tiempo entre cómputo y comunicación. Considérese la configuración con $S = 1$: el tiempo de ejecución en la GPU fue de 2478.59 ms, mientras que la copia *Device* $\rightarrow$ *Host* requirió únicamente 60.97 ms. Esto significa que la fase de transferencia representa apenas un **2,4 %** del tiempo total:

$$\frac{T_{\text{copia}}}{T_{\text{total}}} = \frac{60,97}{2539,56} \approx 0,024 = 2,4\%. \quad (6)$$

En consecuencia, el cómputo neto domina aproximadamente el **97.6 %** del tiempo de ejecución. Esta proporción se mantiene esencialmente constante a través de todas las configuraciones de S .

El cálculo de la matriz de covarianza es un problema **estrictamente *compute-bound***: la complejidad aritmética del producto exterior acumulado $\mathcal{O}(N \cdot D^2)$ impone un volumen de operaciones de punto flotante que satura las unidades de cómputo de la GPU durante la práctica totalidad del tiempo de ejecución. En este régimen, el solapamiento de transferencias con cómputo —la premisa fundamental del *latency hiding*— aporta márgenes de mejora **insignificantes**, ya que el tiempo ocultable (la transferencia PCIe) es un orden de magnitud inferior al tiempo irreducible de cómputo.

Esto explica la forma esencialmente **plana** de la curva de *speedup* de la Figura 2: los valores oscilan entre 0.95 y 1.05, fluctuaciones atribuibles a la variabilidad inherente del *scheduler* de la GPU y del sistema operativo, y no a una tendencia sistemática de mejora o degradación.

5.2. Overhead de la infraestructura de streams

El perfil de Nsight Systems (Tabla 5) revela que las llamadas de infraestructura asociadas a la orquestación por *streams* representan una fracción considerable del tiempo total de la API CUDA. En particular, `cudaHostAlloc` consume 121.93 ms (26.5 %), ya que la asignación de memoria *page-locked* implica que el sistema operativo debe marcar las páginas como no paginables, un proceso costoso que se ejecuta una sola vez pero que no puede solaparse con cómputo. Por su parte, `cudaStreamSynchronize` acumula 113.12 ms (24.6 %), correspondiente a los puntos de barrera en los que el host espera la finalización de cada lote en su respectivo *stream*.

Conjuntamente, estas dos llamadas suman más del **51 %** del tiempo registrado en la API, constituyendo un overhead de gestión que no aporta cómputo útil pero que es inherente a la arquitectura de *streams* asíncronos.

5.3. Crecimiento cuadrático empírico

El paso de 64×48 ($n_1 = 9\,216$) a 96×72 ($n_2 = 20\,736$) en el Experimento 1 permite verificar empíricamente el crecimiento cuadrático esperado por la complejidad $\mathcal{O}(D^2)$:

$$\frac{n_2^2}{n_1^2} = \frac{20\,736^2}{9\,216^2} \approx 5,06 \quad \text{vs.} \quad \frac{T_2}{T_1} = \frac{1\,007,95}{100,79} \approx 10,0. \quad (7)$$

La relación empírica de tiempos ($\approx 10\times$) supera la predicción puramente cuadrática ($\approx 5\times$), lo que sugiere la presencia de factores adicionales como la presión sobre la caché L2, el incremento de la fragmentación de *tiles* y el crecimiento de la copia *Device* $\rightarrow$ *Host* proporcional a D^2 .

5.4. Mitigación del bus PCIe y la VRAM: éxito arquitectónico

Si bien el *latency hiding* no logró reducir el tiempo total de ejecución —resultado que, como se argumentó, era previsible dada la naturaleza *compute-bound* del problema—, la orquestación asíncrona por lotes constituye un **éxito arquitectónico absoluto** en un aspecto igualmente crítico: la **mitigación del límite de la VRAM**.

La implementación tradicional con Stream 0 falla irremediablemente con un error de **Out of Memory** al intentar alojar simultáneamente la totalidad de los datos y la matriz de covarianza en los 3715 MB de VRAM disponibles. Esta limitación impone un techo inamovible sobre la resolución y el número de imágenes procesables.

La arquitectura de *streams* con particionamiento en lotes elimina por completo esta restricción. Al procesar los datos en fragmentos de tamaño controlado, el consumo pico de VRAM queda acotado por el tamaño de un único lote más la matriz de covarianza acumulada, independientemente del volumen total del dataset. En términos prácticos, esto significa que el sistema puede procesar volúmenes de datos **arbitrariamente grandes** —limitados únicamente por la memoria del host y el tiempo de cómputo— sin incurrir en errores de asignación de memoria en la GPU.

La combinación de `cudaMallocHost` (memoria *pinned*) y `cudaMemcpyAsync` garantiza además que las transferencias entre *Host* y *Device* se realicen de la forma más eficiente posible, evitando copias intermedias y utilizando directamente el motor DMA del controlador PCIe.

5.5. Síntesis del análisis

El resultado global puede sintetizarse en dos proposiciones complementarias:

1. **El solapamiento de transferencias es irrelevante para problemas *compute-bound*.** Cuando el tiempo de cómputo supera en uno o más órdenes de magnitud al tiempo de comunicación, el *latency hiding* no puede producir aceleraciones perceptibles. La curva de *speedup* plana confirma experimentalmente esta predicción teórica.

2. **La orquestación por lotes resuelve una limitación arquitectónica fundamental.** Independientemente de su impacto en la velocidad, el particionamiento permite escalar el procesamiento más allá de los límites físicos de la VRAM, transformando un problema intratable en uno viable. Este beneficio es ortogonal al *speedup* y representa la contribución principal de la implementación con *streams*.

6. Conclusiones

El presente trabajo implementó y evaluó dos estrategias para el cálculo de la matriz de covarianza sobre el dataset DIV2K utilizando CUDA: una implementación tradicional sobre el Stream 0 y una implementación con orquestación asíncrona mediante múltiples CUDA Streams.

Los resultados experimentales demuestran que:

- El cálculo de la matriz de covarianza es un problema ***compute-bound***, donde el cómputo del producto exterior acumulado consume aproximadamente el 97.6 % del tiempo total de ejecución. En este régimen, el solapamiento de transferencias PCIe con cómputo no produce mejoras significativas en el tiempo de ejecución, como lo evidencia una curva de *speedup* esencialmente plana ($S_p \approx 1,0$) para $S \in \{1, 2, 4, 8, 16\}$.
- La implementación tradicional con asignación monolítica de memoria falla por **Out of Memory** al escalar la resolución de las imágenes, imponiendo un límite superior rígido sobre el volumen de datos procesable en una GPU con 3715 MB de VRAM.
- La orquestación por lotes mediante CUDA Streams, combinada con *pinned memory* y transferencias asíncronas, resuelve esta limitación de forma definitiva. Al acotar el consumo pico de VRAM al tamaño de un único lote, se habilita el procesamiento de datasets arbitrariamente grandes sin modificar el hardware subyacente.
- Desde una perspectiva de ingeniería de sistemas, la contribución principal de la arquitectura de *streams* no es la aceleración temporal, sino la **viabilidad del particionamiento**: convertir un problema limitado por la memoria física en uno limitado únicamente por el tiempo de cómputo disponible.

Estos hallazgos subrayan la importancia de caracterizar correctamente el *bottleneck* de un algoritmo antes de diseñar estrategias de optimización. En problemas dominados por el cómputo, las técnicas de *latency hiding* sobre el bus PCIe son ineficaces para reducir el tiempo total; no obstante, la infraestructura de *streams* y transferencias asíncronas sigue siendo indispensable como mecanismo de gestión de memoria, habilitando la escalabilidad del sistema más allá de las restricciones físicas del acelerador.